

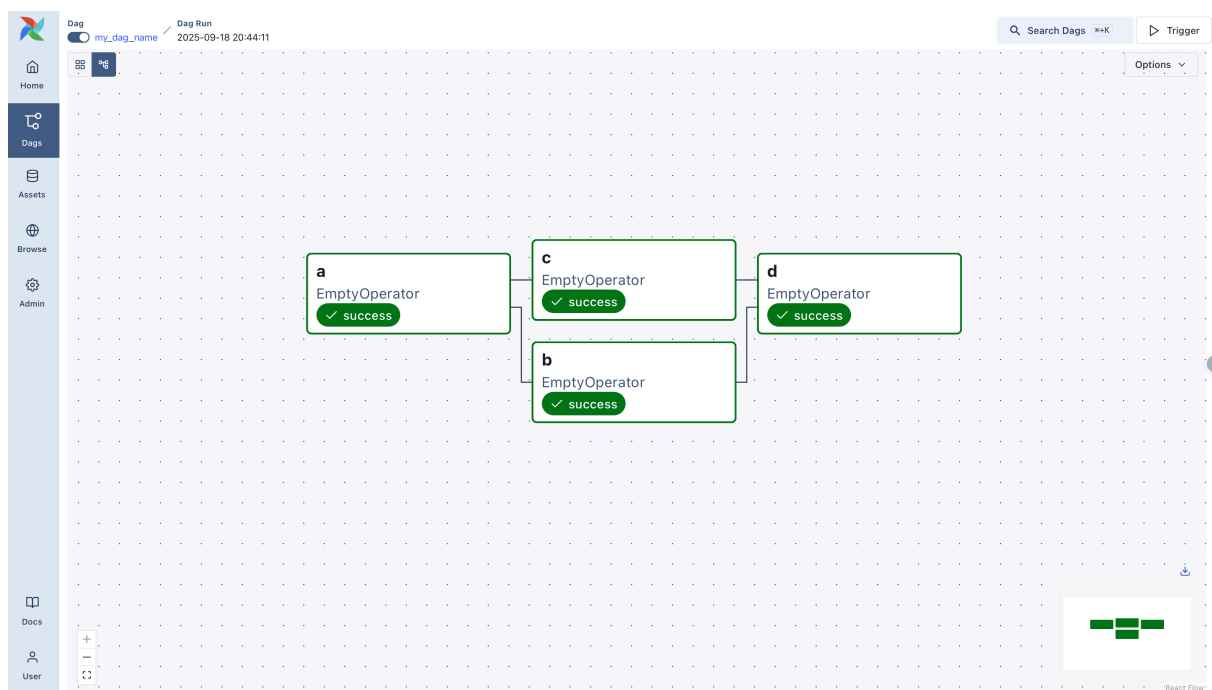
# Airflow

## Dags

Un **Dag** est un modèle qui encapsule tout ce qui est nécessaire pour exécuter un workflow. Certains attributs d'un Dag incluent :

- **Schedule** : moment où le workflow doit être exécuté
- **Tasks** : unités de travail exécutées par des workers
- **Task Dependencies** : ordre et conditions d'exécution des tasks
- **Callbacks** : actions déclenchées à la fin de l'exécution du workflow
- **Additional Parameters** : autres paramètres opérationnels

## Exemple de Dag



Il définit quatre Tasks - A, B, C et D - et détermine l'ordre d'exécution ainsi que les dépendances entre elles. Il définit aussi la fréquence d'exécution du Dag, par exemple : "toutes les 5 minutes à partir de demain" ou "chaque jour depuis le 1er janvier 2020".

Le Dag ne s'intéresse pas au contenu des tasks ; il définit uniquement comment les exécuter : ordre, retries, timeouts, etc.

## Note

Le terme **DAG** vient du concept mathématique "*directed acyclic graph*". Dans Airflow, son sens a évolué au-delà de la structure mathématique stricte. C'est pourquoi le terme **Dag** est utilisé.

## Déclaration d'un Dag

Il existe trois façons de déclarer un Dag :

### 1. Context manager ( **with** statement)

```
import datetime

from airflow.sdk import DAG
from airflow.providers.standard.operators.empty import EmptyOperator

with DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
):
    EmptyOperator(task_id="task")
```

### 2. Constructor classique

```
import datetime

from airflow.sdk import DAG
from airflow.providers.standard.operators.empty import EmptyOperator

my_dag = DAG(
    dag_id="my_dag_name",
```

```
start_date=datetime.datetime(2021, 1, 1),
schedule="@daily",
)

EmptyOperator(task_id="task", dag=my_dag)
```

### 3. Décorateur @dag

```
import datetime

from airflow.sdk import dag
from airflow.providers.standard.operators.empty import EmptyOperator

@dag(start_date=datetime.datetime(2021, 1, 1), schedule="@daily")
def generate_dag():
    EmptyOperator(task_id="task")

generate_dag()
```

Les Dags n'existent pas sans Tasks, qui prennent généralement la forme d'**Operators**, **Sensors** ou **TaskFlow**.

## Running Dags

Dags peuvent être exécutés de deux façons :

- **Déclenchement manuel ou via API**
- **Exécution planifiée (schedule)** définie dans le Dag

Un Dag peut ne pas avoir de schedule, mais c'est très courant d'en définir un.

## Définir un schedule

```
with DAG("my_daily_dag", schedule="@daily"):
    ...
```

## Exemples de schedules

```
with DAG("my_daily_dag", schedule="0 0 * * *"):
    ...
```

```
with DAG("my_one_time_dag", schedule="@once"):
    ...
```

```
with DAG("my_continuous_dag", schedule="@continuous"):
    ...
```

## Cron & Time Intervals

Le schedule d'un **Dag** peut être défini de trois façons :

- une **cron expression**
- un objet `datetime.timedelta`
- un **Cron Preset**

## Exemples

```
from airflow.sdk import DAG
import datetime

dag = DAG("regular_interval_cron_example", schedule="0 0 * * *", ...)
```

```
dag = DAG("regular_interval_cron_preset_example", schedule="@daily", ...)
```

```
dag = DAG("regular_interval_timedelta_example", schedule=da
tetime.timedelta(days=1), ...)
```

## Cron Presets

Pour des besoins de scheduling plus avancés, il est possible d'utiliser un **custom timetable**.

Airflow utilise la librairie **croniter**, qui supporte une syntaxe étendue des cron expressions.

Exemple :

- `0 0 * * MON#1` → premier lundi du mois à 00:00

### Tip

Outils utiles pour écrire des cron expressions :

- [Crontab guru](#)

## Cron presets disponibles

| Preset                   | Meaning                                        |
|--------------------------|------------------------------------------------|
| <code>None</code>        | Pas de schedule (exécution externe uniquement) |
| <code>@once</code>       | Exécution unique                               |
| <code>@continuous</code> | Exécution continue dès la fin du run précédent |
| <code>@hourly</code>     | Toutes les heures (à la fin de l'heure)        |
| <code>@daily</code>      | Tous les jours à minuit                        |
| <code>@weekly</code>     | Chaque semaine le dimanche à minuit            |
| <code>@monthly</code>    | Le 1er jour de chaque mois à minuit            |
| <code>@quarterly</code>  | Tous les 3 mois au premier jour                |
| <code>@yearly</code>     | Le 1er janvier à minuit                        |

Chaque **Dag** est instancié pour chaque exécution planifiée, et chaque exécution crée un **Dag Run** dans la base de données.

# Asset-Aware Scheduling

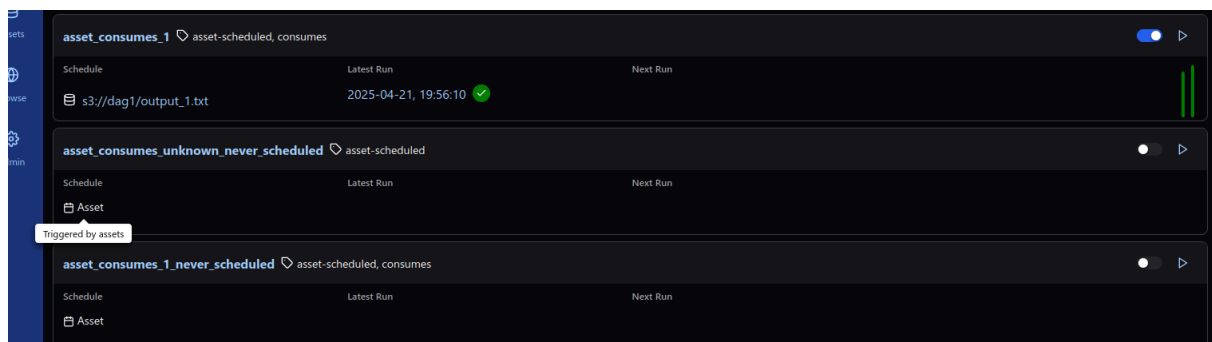
Introduit en version **2.4**

En plus du scheduling basé sur le temps, un **Dag** peut être déclenché lorsqu'un **task met à jour un asset**.

```
from airflow.sdk import DAG, Asset

with DAG(...):
    MyOperator(
        # this task updates example.csv
        outlets=[Asset("s3://asset-bucket/example.csv")],
        ...,
    )
```

```
with DAG(
    # this Dag runs when example.csv is updated
    schedule=[Asset("s3://asset-bucket/example.csv")],
    ...,
):
    ...
```



## Voir aussi

- Asset Definitions pour déclarer des assets

## Scheduling basé sur des assets

Les **assets** permettent de définir des dépendances de données entre Dags.

Exemple :

- le Dag **producer** met à jour un asset
- le Dag **consumer** est déclenché automatiquement après mise à jour

```
example_asset = Asset("s3://asset/example.csv")

with DAG(dag_id="producer", ...):
    BashOperator(task_id="producer", outlets=[example_asset], ...)
```

```
with DAG(dag_id="consumer", schedule=[example_asset], ...):
    ...
```

Règle importante :

- un asset est marqué comme mis à jour uniquement si le task réussit
- si le task échoue ou est skipped → aucun trigger

## Multiple assets

Le paramètre `schedule` peut être une liste d'assets.

Le Dag s'exécute uniquement lorsque **tous les assets ont été mis à jour au moins une fois depuis la dernière exécution.**

```
with DAG(
    dag_id="multiple_assets_example",
    schedule=[
        example_asset_1,
        example_asset_2,
        example_asset_3,
    ],
    ...,
):
    ...
```

Si un asset est mis à jour plusieurs fois avant que tous les autres le soient, le Dag downstream ne s'exécute **qu'une seule fois**.



## Tasks

Un **Task** est l'unité de base d'exécution dans Airflow.

Les tasks sont organisées dans des **Dags**, puis reliées par des dépendances **upstream / downstream** afin de définir leur ordre d'exécution.

## Types de Tasks

Il existe trois types principaux de tasks :

- **Operators** : templates de tasks prédéfinis permettant de construire rapidement un Dag
- **Sensors** : sous-classe d'Operators, dédiés à l'attente d'un événement externe
- **TaskFlow** `@task` : fonction Python décorée transformée en Task

En interne, tous sont des sous-classes de **BaseOperator**.

Les notions de *Task* et *Operator* sont proches, mais :

- **Operators / Sensors** = templates
- **Task** = instance exécutée dans un Dag

## Task Instances

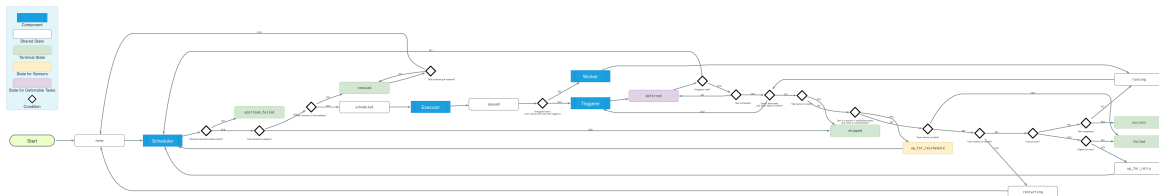
Un **Dag Run** instancie un Dag, et chaque Task devient une **Task Instance**.

Une Task Instance représente une exécution spécifique d'une Task pour un Dag Run donné (donc pour un intervalle de données précis). Elle porte un **state** décrivant son état d'exécution.

## États possibles d'une Task Instance



- **none** : task non encore planifiée (dépendances non satisfaites)
- **scheduled** : planifiée par le scheduler
- **queued** : assignée à un executor, en attente d'un worker
- **running** : en cours d'exécution
- **success** : exécutée sans erreur
- **restarting** : redémarrage demandé pendant l'exécution
- **failed** : échec d'exécution
- **skipped** : ignorée (branching, LatestOnly, etc.)
- **upstream\_failed** : dépendance échouée et bloquante
- **up\_for\_retry** : échec mais retry possible
- **up\_for\_reschedule** : sensor en mode reschedule
- **deferred** : task différée vers un trigger
- **removed** : supprimée du Dag après le début du run



Idéalement, une Task Instance suit le flux :

**none** → **scheduled** → **queued** → **running** → **success**

## Operators

Un **Operator** est un template conceptuel de Task prédéfinie, que l'on déclare directement dans un Dag.

```
with DAG("my-dag") as dag:
    ping = HttpOperator(endpoint="http://example.com/updat
e/")
```

```
email = EmailOperator(to="admin@example.com", subject
="Update complete")

ping >> email
```

Airflow fournit un large ensemble d'Operators, intégrés au core ou disponibles via les providers.

## Operators du core (exemples)

- **BashOperator** : exécute une commande bash
- **PythonOperator** : appelle une fonction Python arbitraire

## TaskFlow API

Le décorateur `@task` permet d'exécuter une fonction Python comme Task.

Il est recommandé à la place de **PythonOperator** classique lorsque les arguments ne nécessitent pas de templating Jinja.

## Note

Le décorateur `@task` est recommandé pour exécuter des fonctions Python simples sans rendu de templates dans les arguments.

## Providers (community operators)

Si un Operator n'est pas inclus par défaut dans Airflow, il est souvent disponible via les **providers**.

## Exemples d'Operators populaires (providers)

- **EmailOperator**
- **HttpOperator**
- **SQLExecuteQueryOperator**
- **DockerOperator**
- **HiveOperator**
- **S3FileTransformOperator**

- **PrestoToMySqlOperator**
- **SlackAPIOperator**

## Sensors

Les **Sensors** sont un type spécial d'**Operator** conçu pour une seule tâche : **attendre qu'un événement se produise**.

Cela peut être :

- un événement basé sur le temps
- l'apparition d'un fichier
- un signal externe

Le sensor reste en attente jusqu'à ce que la condition soit remplie, puis réussit pour permettre l'exécution des tasks downstream. Si la condition n'est pas remplie dans le **timeout**, il échoue et peut déclencher des alertes.

---

## Modes d'exécution

Les Sensors ont deux modes principaux :

- **poke (default)** : occupe un worker slot pendant toute la durée d'attente
- **reschedule** : libère le worker slot entre les vérifications

Le choix dépend du compromis entre ressources et latence :

- checks fréquents (ex: chaque seconde) → **poke**
  - checks espacés (ex: chaque minute) → **reschedule**
- 

## BaseSensorOperator

Tous les sensors héritent de **BaseSensorOperator**, qui définit le comportement commun de polling et de gestion des ressources.

---

## Paramètres principaux

- **poke\_interval** : temps entre deux checks

- **timeout** : durée maximale d'attente avant échec
- **mode** : `poke` ou `reschedule`
- **soft\_fail** : transforme l'échec en `SKIPPED` au lieu de `FAILED`
- **exponential\_backoff** : augmente progressivement le délai entre checks
- **max\_wait** : limite maximale du backoff

---

Référence API :

<https://airflow.apache.org/docs/task-sdk/stable/api.html#airflow.sdk.BaseSensorOperator>

---

## Exemple

```
BashSensor(  
    task_id="wait_for_file",  
    bash_command="test -f /data/input.csv",  
    poke_interval=60,  
    timeout=60 * 60,  
    mode="reschedule",  
)
```

## Tip

Pour les fichiers, utiliser de préférence **FileSensor**, optimisé pour la détection de fichiers et compatible avec le mode déferé (deferrable).

---

## Sensors courants

- **FileSensor** : attend l'apparition d'un fichier
  - **BashSensor** : attend le succès d'une commande bash
  - **PythonSensor** : attend une fonction Python
  - **TimeSensor** : attend une heure spécifique
  - **TimeDeltaSensor** : attend une durée
  - **ExternalTaskSensor** : attend un task d'un autre Dag
-

# TaskFlow

Introduit en version **2.0**.

Le **TaskFlow API** permet d'écrire des Dags en Python "pur" en réduisant le boilerplate via le décorateur `@task`.

## Principe

TaskFlow :

- gère automatiquement les échanges de données via **XComs**
- déduit automatiquement les **dependencies**
- transforme les retours de fonctions en **XComArg**
- permet de chaîner Tasks de manière naturelle

## Exemple

Dans cet exemple :

- `get_ip` → TaskFlow task qui retourne une valeur
- `compose_email` → dépend automatiquement de `get_ip`
- `send_email_notification` → Operator classique utilisant les résultats précédents

Airflow :

- stocke les retours via **XCom**
- crée automatiquement les **dependencies** entre tasks

```
from airflow.sdk import task
from airflow.providers.smtp.operators.smtp import EmailOperator
```

```
@task
def get_ip():
    return my_ip_service.get_main_ip()
```

```
@task(multiple_outputs=True)
def compose_email(external_ip):
    return {
        "subject": f"Server connected from {external_ip}",
        "body": f"Your server is connected from {external_ip}"
```

```

    }

    email_info = compose_email(get_ip())

    EmailOperator(
        task_id="send_email_notification",
        to="example@example.com",
        subject=email_info["subject"],
        html_content=email_info["body"],
    )

```

## XComArg

Lorsqu'une TaskFlow function est appelée dans un Dag, elle ne s'exécute pas immédiatement.

Elle retourne un objet **XComArg**, représentant la valeur future de la task.

## Passage de valeurs

Les valeurs peuvent être passées directement :

```

@task
def hello_name(name: str):
    print(f"Hello {name}!")

hello_name("Airflow users")

```

## Résultat

- Simplification de la création de Dags
- Moins de boilerplate
- Dépendances automatiques

- Intégration native avec Operators existants

## Installation de Apache Airflow avec Docker

### 1. Télécharger le fichier Docker Compose

```
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/3.2.1/docker-compose.yaml'
```

### 2. Créer les dossiers nécessaires

```
mkdir -p ./dags ./logs ./plugins ./config
```

Rôle des dossiers :

- `dags` → workflows Airflow
- `logs` → journaux d'exécution
- `plugins` → extensions personnalisées
- `config` → configuration additionnelle

### 3. Définir l'utilisateur Linux

```
echo -e "AIRFLOW_UID=$(id -u)" > .env
```

### 4. Vérifier la configuration Airflow

```
docker compose run airflow-cli airflow config list
```

### 5. Initialiser Airflow

```
docker compose up airflow-init
```

Identifiants par défaut :

- utilisateur : `airflow`
  - mot de passe : `airflow`
- 

## 6. Démarrer Airflow

```
docker compose up
```

Ou en arrière-plan :

```
docker compose up -d
```

---

## 7. Accéder à l'interface web

Ouvrir :

```
http://localhost:8080
```

---